OBSTACLE AVOIDING CAR PROJECT REPORT

Title: Design and Implementation of an Obstacle Avoiding Car Using Arduino

Abstract:

This project presents the design and development of an obstacle avoiding robotic car using an Arduino microcontroller and ultrasonic sensor. The system detects obstacles in its path and automatically changes direction to avoid collisions. The robot operates autonomously without human intervention and demonstrates basic robotics concepts such as sensing, decision-making, and motion control. This project is useful for applications like automated vehicles, surveillance systems, and robotics learning.

Introduction:

Obstacle avoiding robots are one of the most fundamental and widely used robotics projects. The main goal is to build a system that can detect obstacles and navigate safely without human control.

This project uses:

- Ultrasonic sensor for distance measurement
- Arduino for processing
- Motor driver for controlling motors

Such systems are used in:

- Self-driving vehicles
- Industrial automation
- Robotics competitions

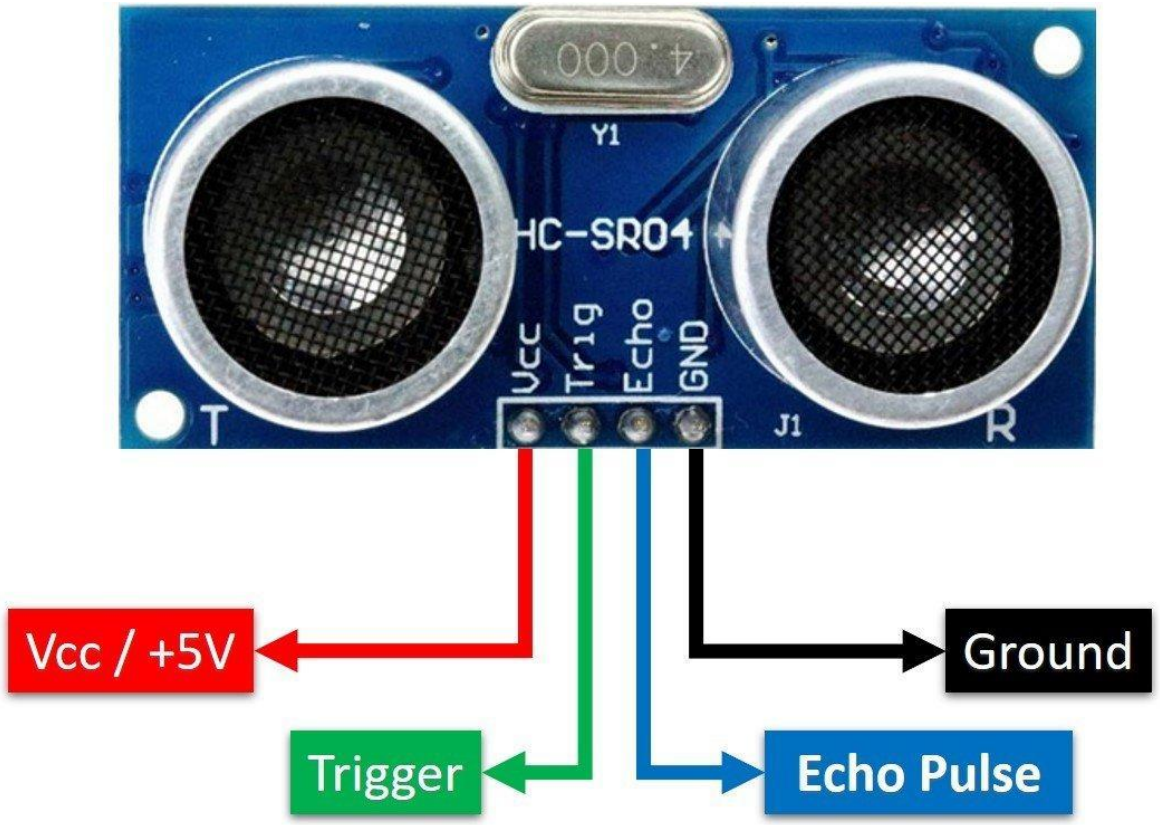
Objectives:

- To design an autonomous obstacle avoiding vehicle
- To understand working of ultrasonic sensors
- To learn motor control using a motor driver
- To implement real-time decision-making

Components Required:

- Arduino Uno
- Ultrasonic Sensor (HC-SR04)
- Motor Driver (L298N)
- DC Motors (4)
- Robot chassis with wheels
- Battery (9V / Li-ion)
- Jumper wires

Working Principle:



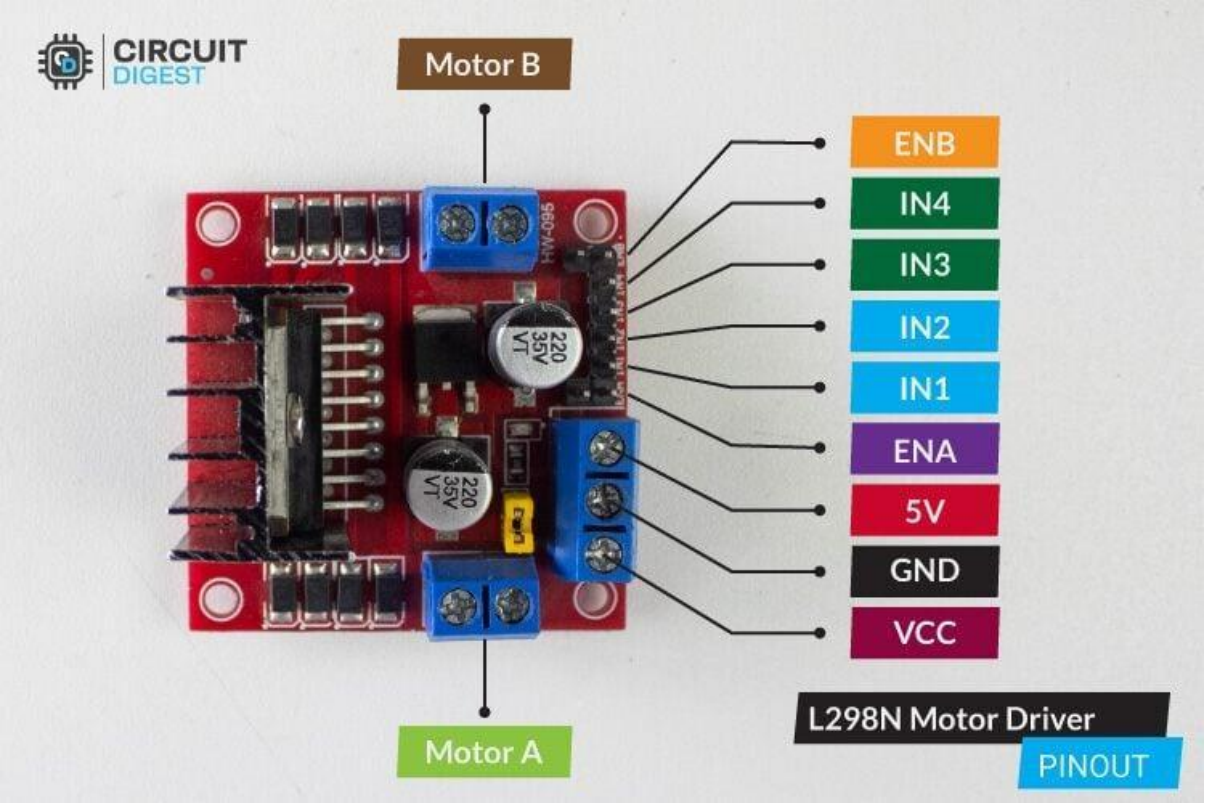
The ultrasonic sensor sends sound waves and measures the time taken for the echo to return.

Distance formula:

$$\text{Distance} = \frac{\text{Speed of Sound} \times \text{Time}}{2}$$

- If distance < threshold → obstacle detected
- If distance > threshold → move forward

Motor Driver (L298N)



- Controls direction of motors
- Receives signals from Arduino
- Allows forward, backward, left, right movement

System Working

1. Sensor continuously measures distance
2. Arduino processes the data
3. If obstacle detected:
    - Stop
    - Turn left/right
4. If path clear:
    - Move forward

7. Circuit Diagram

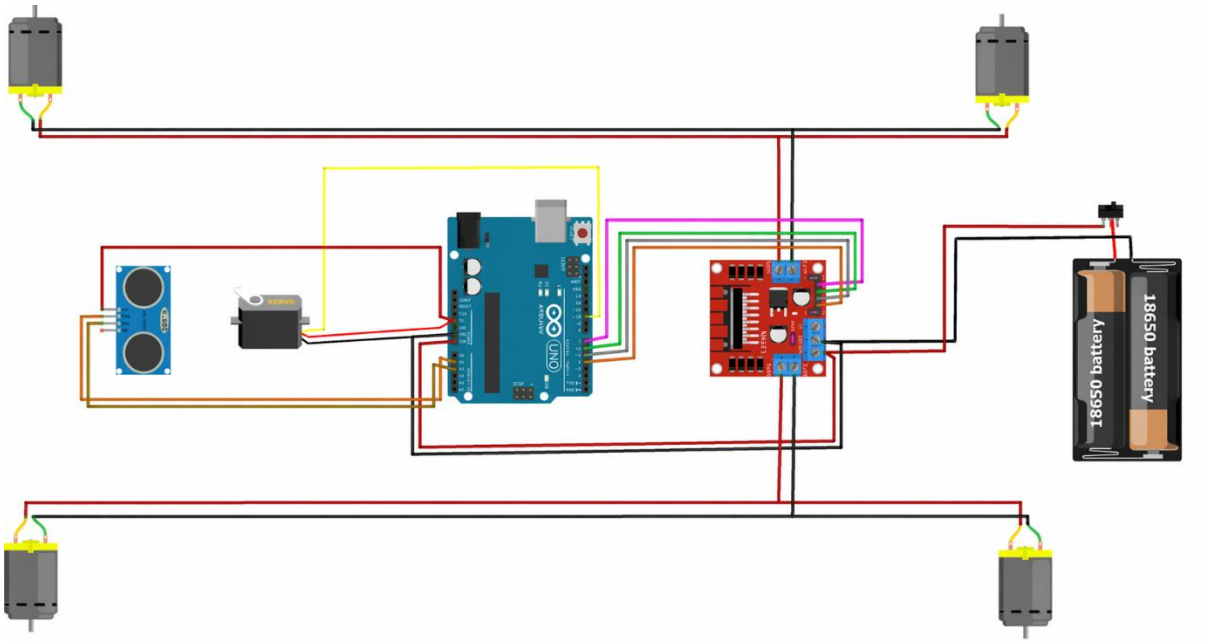


Fig7.1. Circuit Diagram

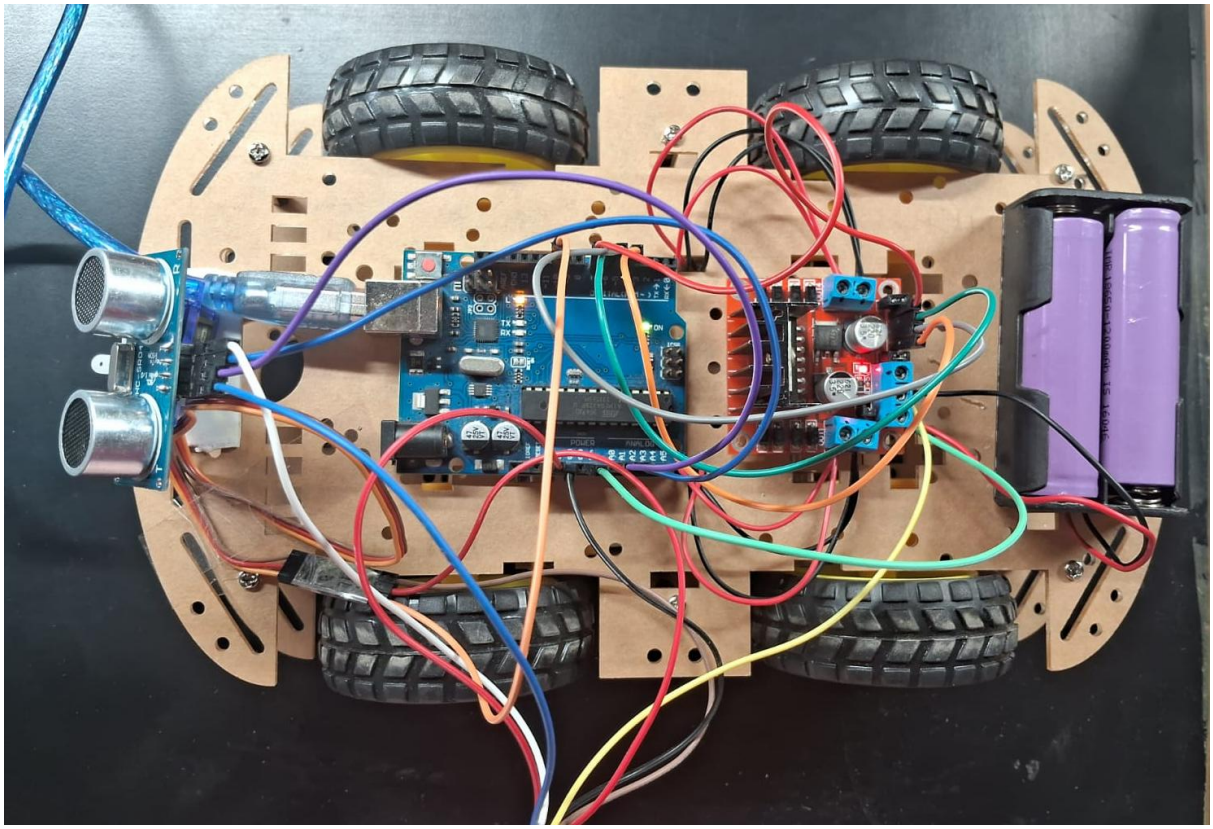


Fig7.2. Hardware Implementation

8. Code (Arduino)

```
#include <Servo.h>          //Servo motor library. This is standard library
#include <NewPing.h>        //Ultrasonic sensor function library. You must install this library

//our L298N control pins
const int LeftMotorForward = 7;
const int LeftMotorBackward = 6;
const int RightMotorForward = 5;
const int RightMotorBackward = 4;

//sensor pins
#define trig_pin A1 //analog input 1
#define echo_pin A2 //analog input 2

#define maximum_distance 200
boolean goesForward = false;
int distance = 100;

NewPing sonar(trig_pin, echo_pin, maximum_distance); //sensor function
Servo servo_motor; //our servo name

void setup(){

  pinMode(RightMotorForward, OUTPUT);
  pinMode(LeftMotorForward, OUTPUT);
  pinMode(LeftMotorBackward, OUTPUT);
  pinMode(RightMotorBackward, OUTPUT);

  servo_motor.attach(10); //our servo pin

  servo_motor.write(115);
  delay(2000);
  distance = readPing();
  delay(100);
  distance = readPing();
  delay(100);
  distance = readPing();
  delay(100);
  distance = readPing();
  delay(100);
}

void loop(){

  int distanceRight = 0;
  int distanceLeft = 0;
  delay(50);

  if (distance <= 45){
```

```
    moveStop();
    delay(300);
    moveBackward();
    delay(400);
    moveStop();
    delay(300);
    distanceRight = lookRight();
    delay(300);
    distanceLeft = lookLeft();
    delay(300);

    if (distance >= distanceLeft){
      turnRight();
      moveStop();
    }
    else{
      turnLeft();
      moveStop();
    }
  }
  else{
    moveForward();
  }
    distance = readPing();
}

int lookRight(){
  servo_motor.write(50);
  delay(500);
  int distance = readPing();
  delay(100);
  servo_motor.write(115);
  return distance;
}

int lookLeft(){
  servo_motor.write(170);
  delay(500);
  int distance = readPing();
  delay(100);
  servo_motor.write(115);
  return distance;
  delay(100);
}

int readPing(){
  delay(70);
  int cm = sonar.ping_cm();
  if (cm==0){
    cm=250;
```

```
  }
  return cm;
}

void moveStop(){

  digitalWrite(RightMotorForward, LOW);
  digitalWrite(LeftMotorForward, LOW);
  digitalWrite(RightMotorBackward, LOW);
  digitalWrite(LeftMotorBackward, LOW);
}

void moveForward(){

  if(!goesForward){

    goesForward=true;

    digitalWrite(LeftMotorForward, HIGH);
    digitalWrite(RightMotorForward, HIGH);

    digitalWrite(LeftMotorBackward, LOW);
    digitalWrite(RightMotorBackward, LOW);
  }
}

void moveBackward(){

  goesForward=false;

  digitalWrite(LeftMotorBackward, HIGH);
  digitalWrite(RightMotorBackward, HIGH);

  digitalWrite(LeftMotorForward, LOW);
  digitalWrite(RightMotorForward, LOW);

}

void turnRight(){

  digitalWrite(LeftMotorForward, HIGH);
  digitalWrite(RightMotorBackward, HIGH);

  digitalWrite(LeftMotorBackward, LOW);
  digitalWrite(RightMotorForward, LOW);

  delay(250);

  digitalWrite(LeftMotorForward, HIGH);
  digitalWrite(RightMotorForward, HIGH);
```

```
  digitalWrite(LeftMotorBackward, LOW);
  digitalWrite(RightMotorBackward, LOW);



}

void turnLeft(){

  digitalWrite(LeftMotorBackward, HIGH);
  digitalWrite(RightMotorForward, HIGH);

  digitalWrite(LeftMotorForward, LOW);
  digitalWrite(RightMotorBackward, LOW);

  delay(250);

  digitalWrite(LeftMotorForward, HIGH);
  digitalWrite(RightMotorForward, HIGH);

  digitalWrite(LeftMotorBackward, LOW);
  digitalWrite(RightMotorBackward, LOW);
}
```

9. Advantages

- Fully autonomous system
- Low cost and simple design
- Easy to implement
- Useful for beginners in robotics

10. Limitations

- Limited detection range
- Cannot detect very soft/angled surfaces properly
- No path optimization (random movement)

11. Applications

- Self-driving vehicles
- Industrial robots
- Surveillance systems
- Automated cleaning robots

12. Result

The obstacle avoiding car successfully detects obstacles and avoids collisions by changing its direction automatically. The system operates efficiently in real-time conditions.

13. Future Scope

- Add AI-based path planning
- Use camera for object detection
- Integrate IoT for remote monitoring
- Add GPS navigation

14. Conclusion

The obstacle avoiding car successfully demonstrates autonomous navigation using an ultrasonic sensor and Arduino. It detects obstacles and changes direction without human input. The project helps understand sensor interfacing, motor control, and basic robotics. It works well in simple environments, with scope for improvement using advanced algorithms and sensors.